

API Gaiola

Documentação Técnica — REST API

Gaiolas / Carrinhos Logística — Tabela ZDT

 Responsável	Daniel Victor da Rosa – Personalitec
 Data de Criação	2026
 Namespace	pda_gaiola
 Arquivo	BUD1521.TLPP
 Tabela	ZDT — Gaiolas / Carrinhos Logística
 Protocolo	REST / HTTP

1. Visão Geral

A API Gaiola (API_GAIOLA.TLPP) é responsável por gerenciar operações de atualização relacionadas às gaiolas e carrinhos logísticos cadastrados no sistema, permitindo a atualização de status e localização física dos itens via requisição REST.

2. Arquitetura de Classes

O fonte foi estruturado seguindo os princípios de Orientação a Objetos, com separação clara de responsabilidades entre entidade e operação.

2.1 Interfaces

Interface	Métodos Definidos	Descrição
GaiolaInterface	New(), getCodigo(), getStatus(), getLocal(), setCodigo(c), setStatus(c), setLocal(c)	Contrato da entidade Gaiola: getters e setters das propriedades privadas.
AtualizarInterface	New(), alterar() as Logical	Contrato da operação de atualização via endpoint REST.

2.2 Classes

Classe	Herança / Implements	Responsabilidade
Gaiola	Implements GaiolaInterface	Entidade principal. Armazena as propriedades privadas (cCodigo, cStatus, cLocal) e expõe getters/setters.
Atualizar	From Gaiola Implements AtualizarInterface	Herda de Gaiola para reutilizar seus métodos e adiciona o endpoint REST PUT.

2.3 Propriedades Privadas — Classe Gaiola

Atributo	Tipo	Descrição
cCodigo	Character	Código da gaiola/carrinho
cStatus	Character	Status atual da gaiola/carrinho
cLocal	Character	Localização física (local físico)

3. Endpoints da API

3.1 Alterar Gaiola/Carrinho

Método HTTP	PUT
Endpoint	pda/api_gaiola/alterar
Descrição	Atualiza o status e a localização física de uma gaiola ou carrinho logístico na tabela ZDT.
Autenticação	Padrão Protheus (sessão REST)

Body da Requisição (JSON)

Campo	Tipo	Obrigatório	Descrição
codigo	String	Sim	Código da gaiola/carrinho (chave de busca na ZDT)
status	String	Sim	Novo status a ser gravado em ZDT_STATUS
local	String	Sim	Nova localização física a ser gravada em ZDT_LOCFIS

Exemplo de Body:

```
{
  "codigo": "0401 ",
  "status": "TESTE",
  "local": "99"
}
```

Campos Gravados — Tabela ZDT

Campo ZDT	Origem (Body)	Descrição
ZDT_STATUS	status	Status da gaiola/carrinho
ZDT_LOCFIS	local	Localização física

Códigos de Retorno

Code	Situação	Mensagem
1	Campo codigo ausente no body	O campo (codigo) nao foi informado!
2	Campo status ausente no body	O campo (status) nao foi informado!
3	Campo local ausente no body	O campo (local) nao foi informado!
4	Registro não encontrado na ZDT	Registro nao encontrado

200	Atualização realizada com sucesso	Gaiola/Carrinho atualizada com sucesso
500	Erro interno no servidor	{error:Internal Server Error}

3.2 Exemplos de Resposta

✓ Sucesso (HTTP 200):

```
{
  "retorno": [
    {
      "message": "Gaiola/Carrinho atualizada com sucesso",
      "code": 200
    }
  ]
}
```

✗ Campo obrigatório ausente (HTTP 200 — erro de negócio):

```
{
  "retorno": [
    {
      "message": "O campo (codigo) nao foi informado!",
      "code": 1
    }
  ]
}
```

⚠ Registro não encontrado (HTTP 200 — erro de negócio):

```
{
  "retorno": [
    {
      "message": "Registro nao encontrado",
      "code": 4
    }
  ]
}
```

● Erro interno (HTTP 500):

```
{
  error: "Internal Server Error"
}
```

4. Fluxo de Processamento — Método alterar()

- | | |
|---|---|
| 1 | Recebe o body JSON da requisição via <code>oRest.getBodyRequest()</code> |
| 2 | Decodifica UTF-8 e carrega o JSON com <code>oBody.FromJson()</code> |
| 3 | Valida presença dos campos obrigatórios: <code>codigo</code> → <code>status</code> → <code>local</code> |
| 4 | Seta os valores na classe via <code>setCodigo()</code> , <code>setStatus()</code> , <code>setLocal()</code> |
| 5 | Seleciona a área ZDT e aplica ordem 1 (<code>DbSetOrder(1)</code>) |
| 6 | Busca o registro com <code>DbSeek(xFilial('ZDT') + cCodigo)</code> |

7	Se encontrado: RecLock → atualiza ZDT_STATUS e ZDT_LOCFIS → MsUnlock
8	Se não encontrado: retorna code 4
9	Serializa a resposta com oRetorno.toJson() + EncodeUtf8 e envia via oRest:setResponse()

5. Observações Técnicas

Herança (OOP): A classe Atualizar herda de Gaiola (From Gaiola), reutilizando seus métodos de get/set sem duplicação de código. Para utilizar somente os dados da gaiola sem a operação REST, instancie diretamente a classe Gaiola.

Encoding: O body da requisição é decodificado com DecodeUTF8() antes do parse JSON, e a resposta é codificada com EncodeUtf8() antes do envio, garantindo suporte a caracteres especiais.

Tratamento de Erros: O bloco BEGIN SEQUENCE / RECOVER garante que, mesmo em caso de exceção inesperada, o endpoint retorna HTTP 500 com mensagem de erro controlada.

Lock de Registro: A gravação utiliza RecLock('ZDT', .F.) para bloqueio de registro existente e MsUnlock() para liberação após a escrita, seguindo o padrão Protheus.

Busca com Filial: A busca na ZDT utiliza xFilial('ZDT') + cCodigo, garantindo o isolamento correto por filial no ambiente multiempresa do Protheus.